Spring Microservices Labs

Vishwas K Singh, SME-FSD

26 November 2024

Contents

1 Lab: Creating Service Registry for Ecommerce App 4

1.1 Tasks . . . 4

1.1.1 Task 1: Generating Template from spring initializer . . . 4

1.1.2 Task 2: Open the project in intellij . . . 4

1.1.3 Task 3: Enable Eureka Server . . . 4

1.1.4 Task 4: Add Configuration in application.yml . . . 5

1.2 Output . . . 5

2 Lab: Creating Product Service for Ecommerce App 7

2.1 Tasks . . . 7

2.1.1 Task 1: Generating Template from spring initializer . . . 7

2.1.2 Task 2: Open the project in intelliJ . . . 7

2.1.3 Task 3: Creating Product Service . . . 7

2.1.4 Task 4: Add product Service as Eureka Client . . . 8

2.2 Output . . . 9

3 Lab: Creating Order Service for Ecommerce App 10

3.1 Tasks . . . 10

3.1.1 Task 1: Generating Template from spring initializer . . . 10

3.1.2 Task 2: Open the project in intelliJ . . . 10

3.1.3 Task 3: Creating Order Service . . . 10

3.1.4 Task 4: Add order Service as Eureka Client . . . 11

3.2 Output . . . 11

4 Lab: Adding a configuration server & externalize config to github 12

4.1 Tasks . . . 12

4.1.1 Task 1: Generating Template from spring initializer . . . 12

4.1.2 Task 2: Open the project in intellij . . . 12

4.1.3 Task 3: Enable Config Server . . . 12

4.1.4 Task 4: Add the configuration to github . . . 13

4.1.5 Task 5: add the configuration to the application.yaml . . . 13

4.1.6 Task 6: Adding config client in product and order service . . . 13

4.2 Output . . . 14

5 Lab: Communication between order service and product service in Ecommerce App 15

5.1 Tasks . . . 15

5.1.1 Task 1: In the Order Service add the dependency open feign client . . . 15

5.1.2 Task 2: Make Order Service as a open feign client . . . 15

5.1.3 Task 3: Adding Product service as external dependency . . . 15

5.1.4 Task 4: In the interface product service add . . . 16

6 Lab: Handling errors from one service to another in Eommerce App 18

6.1 Tasks . . . 18

6.1.1 Task 1: In the external package – create a package decoder . . . 18

6.1.2 Task 2: Create a class custom error decoder which implements ErrorDecoder from feign client . . . 18
6.1.3 Task 3: Add config to use custom error decoder instead of error decoder . . . 19
6.1.4 Task 4: Adding Exception Handling in order service . . . 20

7 Lab: Add zipkin tracing for Ecommerce Application 22
7.1 Tasks . . . 22

8 Lab: Adding Payment service to Ecommerce App 24
8.1 Tasks . . . 24
8.1.1 Task 1: Generating Template from spring initializer . . . 24
8.1.2 Task 2: Open the project in intelliJ . . . 24
8.1.3 Task 3: Create Payment Service . . . 25

9 Lab: Communication among services using RestTemplate 28
9.1 Tasks . . . 28
9.1.1 Task 1: In the Service you want to fetch data `ORDER SERVICE` . . . 28
9.1.2 Task 2: In the external package . . . 28
9.1.3 Task 5: Create Payment service openfeign client . . . 28
9.1.4 Task 4: Modify PlaceOrder Method to updated payment . . . 29
9.1.5 Task 5: Implement getorder details . . . 30
9.2 Output . . . 31

10 Lab: Implementing Logging in Ecommerce Application 32
10.1 Tasks . . . 32
10.1.1 Task 1: Enable Log4J2 in application . . . 32
10.1.2 Task 2: Trigger endpoint . . . 32
10.2 Output . . . 32

11 Lab: Adding API Gateway to Ecommerce Application 34
11.1 Tasks . . . 34
11.1.1 Task 1: Generate template . . . 34
11.1.2 Task 2: Add the dependencies . . . 34
11.1.3 Task 3: Open in IntelliJ . . . 34
11.1.4 Task 4: Add the routes . . . 34

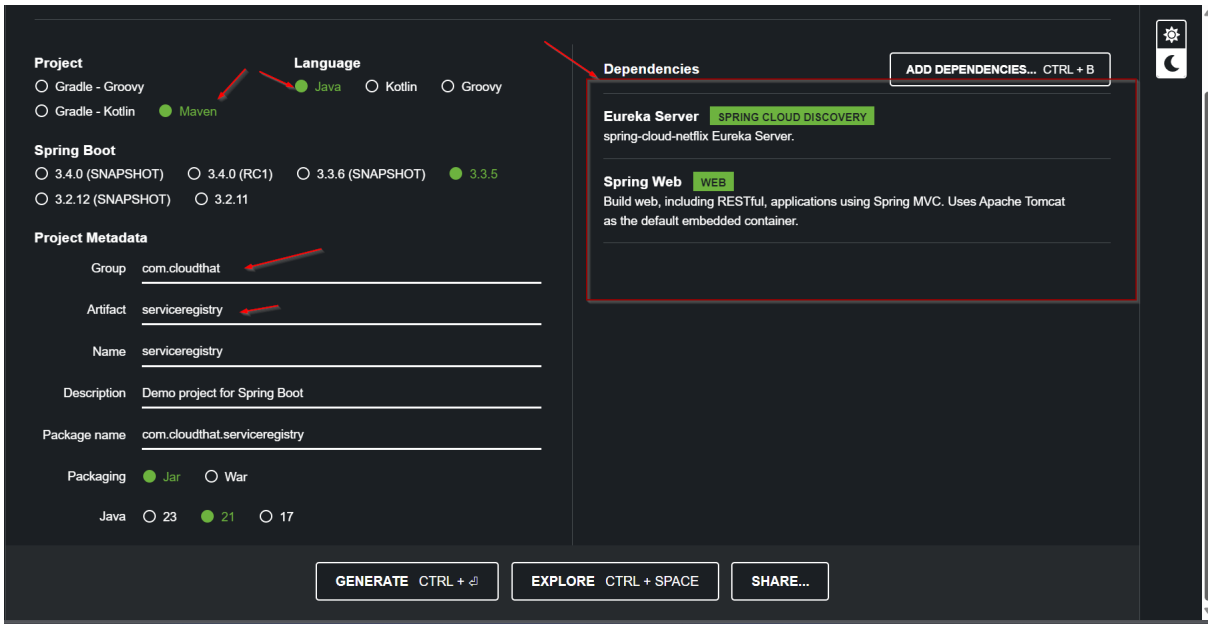
12 Lab: Implementing Circuit Breaker 36
12.1 Tasks . . . 36
12.1.1 Task 1: Adding dependency to cloud gateway . . . 36
12.1.2 Task 2: Add fallback uri . . . 36
12.1.3 Task 3: Implement Fallback controller . . . 36
12.1.4 Task 4: In cloudgateway – create bean for circuit breaker . . . 37
12.1.5 Task 5: Adding Circuitbreaker . . . 37
12.1.6 Task 6: Add configuration in order service . . . 38
12.1.7 Task 7: Implementing Rate Limiter . . . 38
12.1.8 Task 8: Add replenish rate and burst capacity . . . 39
12.2 Output . . . 40

1 Lab: Creating Service Registry for Ecommerce App

1.1 Tasks

1.1.1 Task 1: Generating Template from spring initializer

- visit the spring initializer[1]
- Add the dependencies



1.1.2 Task 2: Open the project in intellij

- Convert `application.properties` to `application.yaml`

1.1.3 Task 3: Enable Eureka Server

- In Application File - `@EnableEurekaServer`

```
package com.cloudthat.serviceregistry;

import org.springframework.boot.SpringApplication;

@SpringBootApplication
@EnableEurekaServer
public class ServiceRegistryApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceRegistryApplication.class, args);
    }

}
```

[1] https://start.spring.io

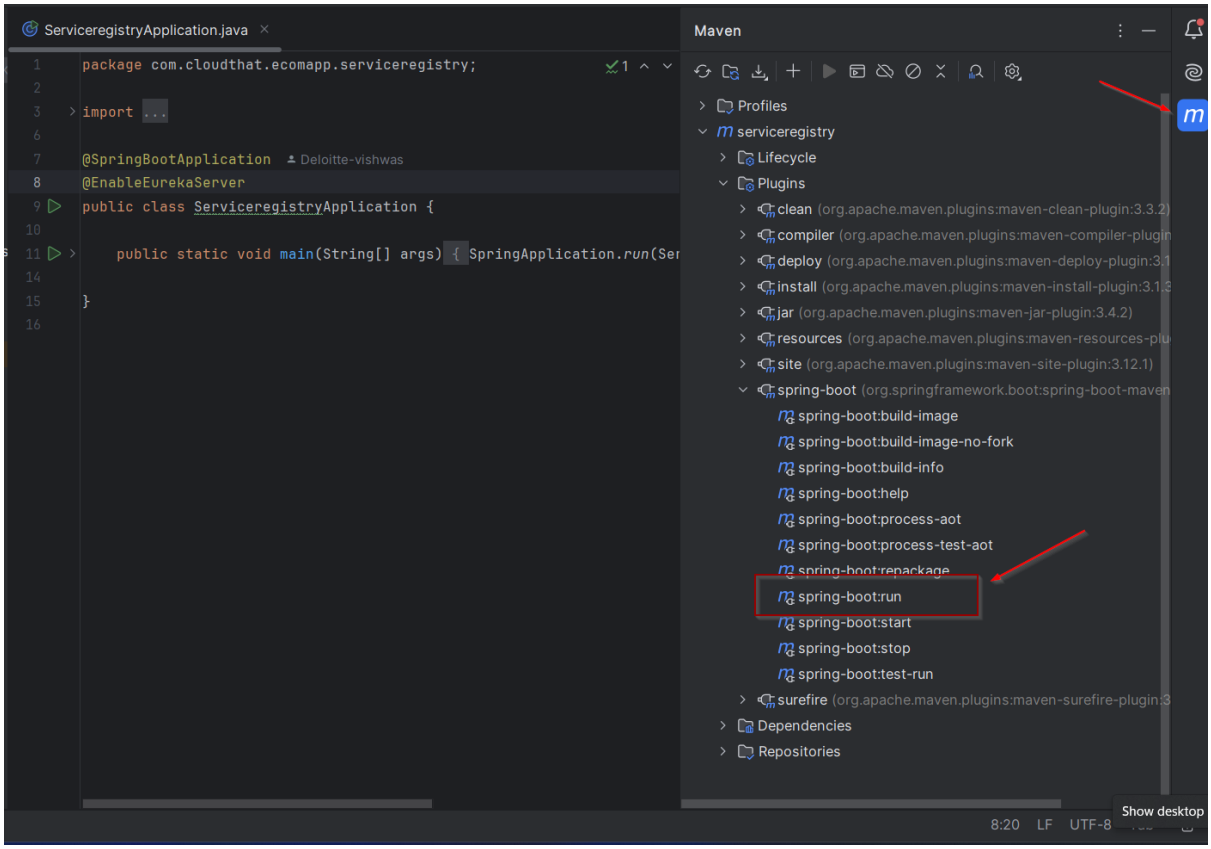
1.1.4 Task 4: Add Configuration in application.yml

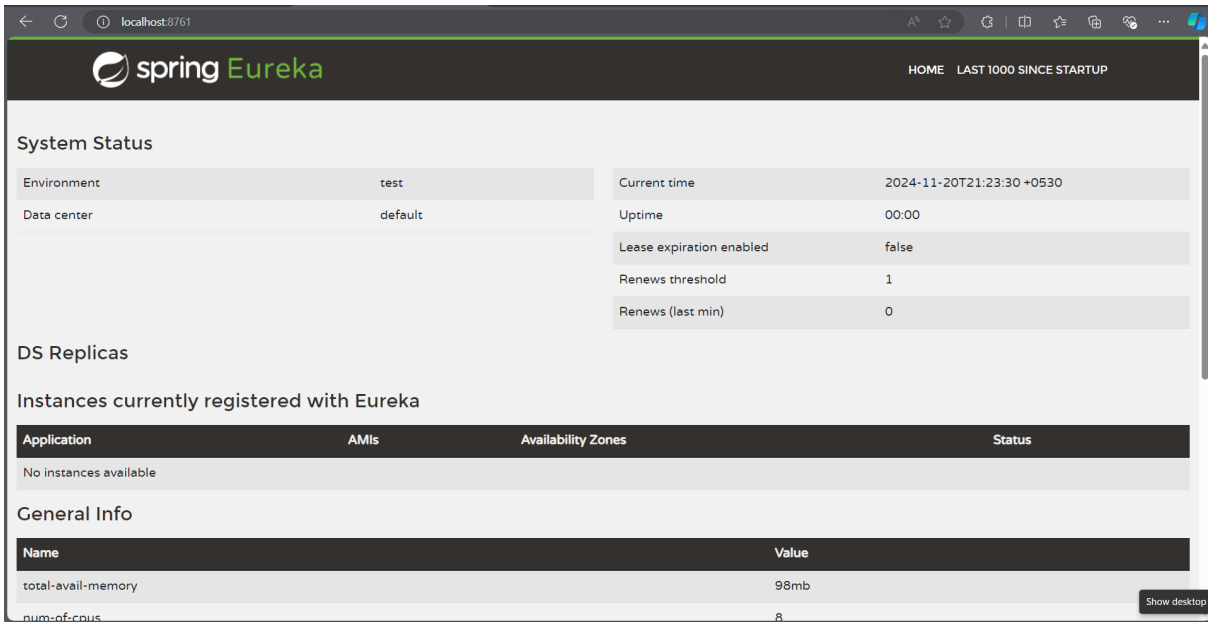
set `server.port` to 8761

```yaml
server:
  port: 8761
spring:
  application:
    name: service-registry
eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://${eureka.instance.hostname}:{server.port}/eureka/
```

1.2 Output

Now you can check the service registry using the above address





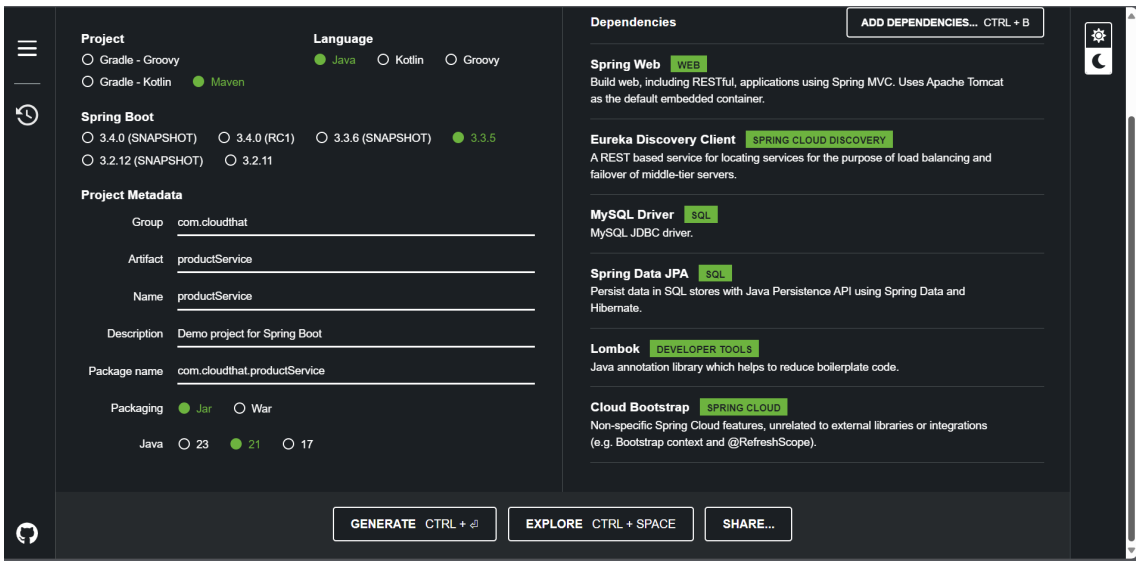
visit https://spring.io/guides/gs/service-registration-and-discovery for more

2 Lab: Creating Product Service for Ecommerce App

2.1 Tasks

2.1.1 Task 1: Generating Template from spring initializer

- visit the spring initializer[2]
- Add the dependencies
  - Spring Web
  - Eureka Client
  - Lombok
  - Data JPA
  - Mysql
  - Cloud Bootstrap



2.1.2 Task 2: Open the project in intelliJ

- Convert `application.properties` to `application.yaml`
- Set `server.port` to `8080`

2.1.3 Task 3: Creating Product Service

- Create – endpoints as in spring boot app as you created earlier
- Product Resource has the fields as below
  - productId
  - productName
  - price
  - quantity
  - category

[2] https://start.spring.io

- Also add an endpoint

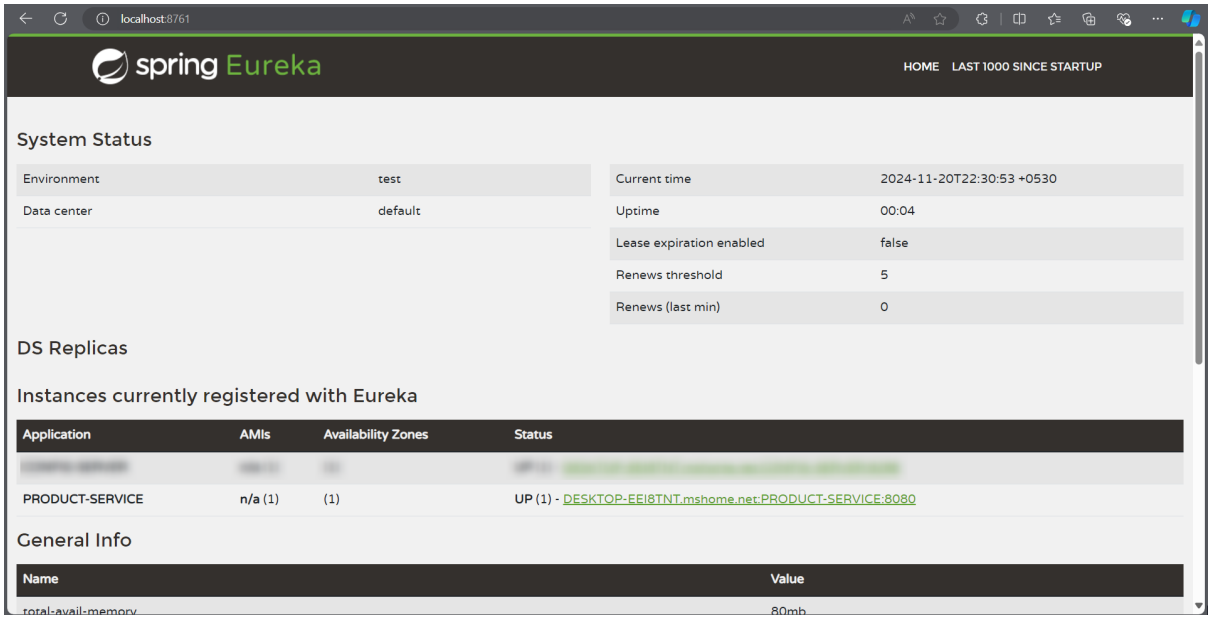
```
@PutMapping("/reduceQuantity/{id}")  no usages
public ResponseEntity<Void> reduceQuantity(
        @PathVariable("id") long productId,
        @RequestParam long quantity
) {
        productService.reduceQuantity(productId,quantity);
        return new ResponseEntity<>(HttpStatus.OK);
}
```

2.1.4 Task 4: Add product Service as Eureka Client

```
server:
  port: 8080

spring:
    datasource:
        driver-class-name: com.mysql.cj.jdbc.Driver
        password: [redacted]
        url: jdbc:mysql://localhost:3306/productdb
        username: [redacted]
    jpa:
        show-sql: true
        hibernate:
            ddl-auto: update
    application:
        name: PRODUCT-SERVICE
[redacted]

eureka:
  instance:
    prefer-ip-address: true
  client:
    fetch-registry: true
    register-with-eureka: true
    service-url:
      defaultZone: ${EUREKA_SERVER_ADDRESS:http://localhost:8761/eureka}
```

test the product service and service registry

2.2 Output

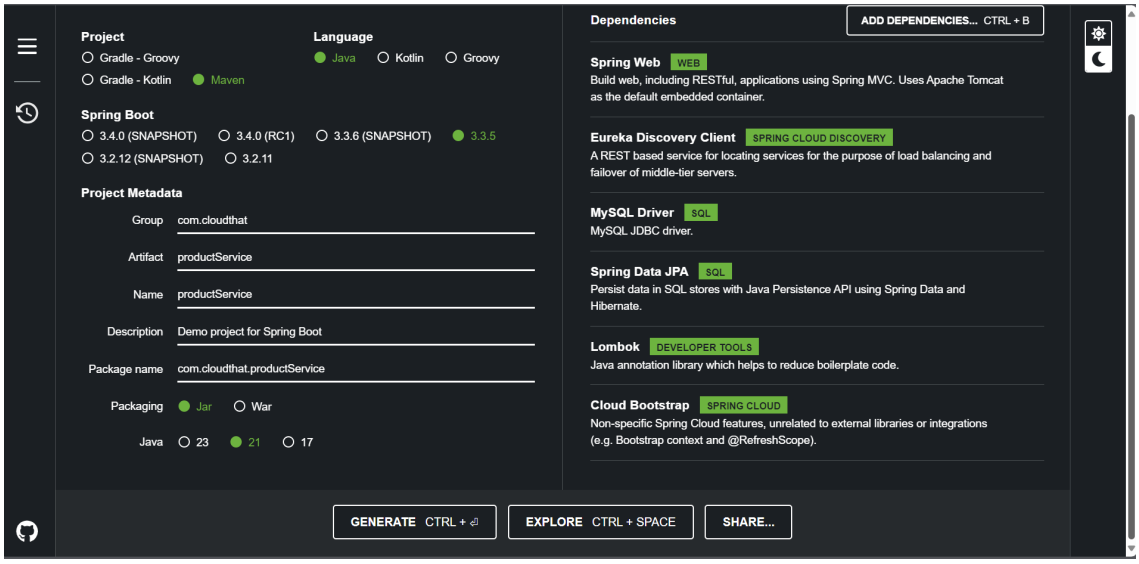

localhost:8761
spring Eureka
HOME
LAST 1000 SINCE STARTUP
System Status
Environment
test
Data center
default
Current time
2024-11-20T22:30:53 +0530
Uptime
00:04
Lease expiration enabled
false
Renews threshold
5
Renews (last min)
0
DS Replicas
Instances currently registered with Eureka
Application
AMIs
Availability Zones
Status
PRODUCT-SERVICE
n/a (1)
(1)
UP (1) - DESKTOP-EEI8TNT.mshome.net:PRODUCT-SERVICE:8080
General Info
Name
Value

3 Lab: Creating Order Service for Ecommerce App

3.1 Tasks

3.1.1 Task 1: Generating Template from spring initializer

- visit the spring initializer[3]
- Add the dependencies
  - Spring Web
  - Eureka Client
  - Lombok
  - Data JPA
  - Mysql
  - Cloud Bootstrap



3.1.2 Task 2: Open the project in intelliJ

- Convert `application.properties` to `application.yaml`
- Set `server.port` to `8082`

3.1.3 Task 3: Creating Order Service

- Create – endpoints as in spring boot app as you created earlier
- Order Resource has the fields as below
  - orderId
  - productId
  - amount
  - quantity
  - orderStatus
  - orderDate

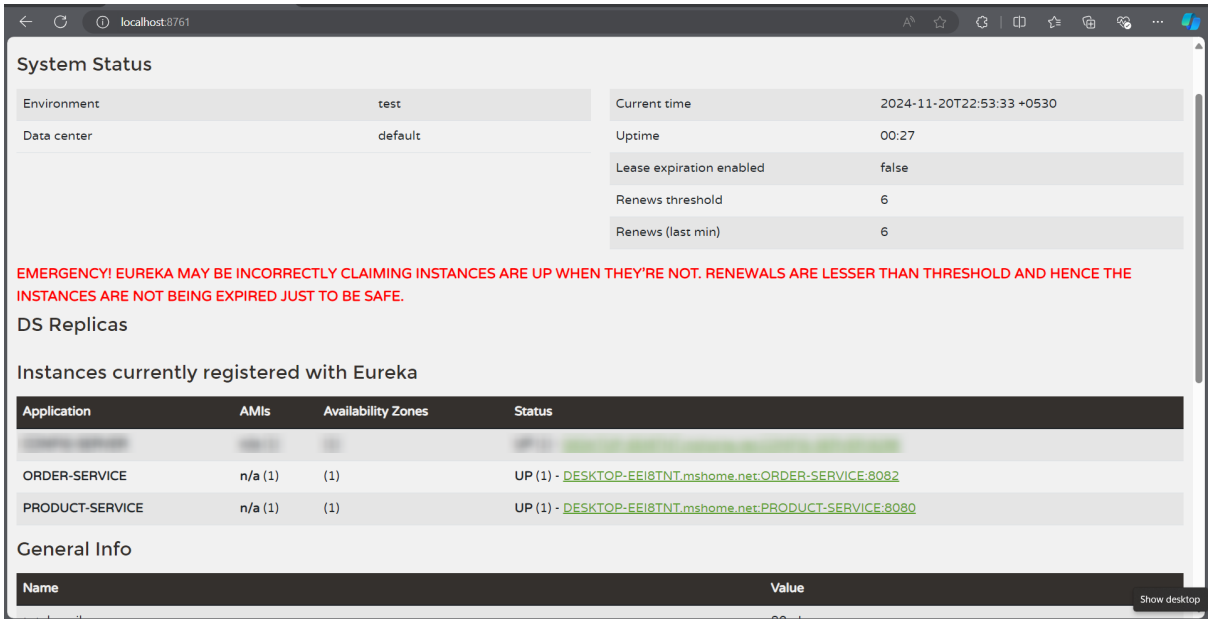
[3] https://start.spring.io

3.1.4 Task 4: Add order Service as Eureka Client

```yaml
server:
  port: 8082

spring:
    datasource:
        driver-class-name: com.mysql.cj.jdbc.Driver
        password: root
        url: jdbc:mysql://localhost:3306/orderdb
        username: root
    jpa:
        show-sql: true
        hibernate:
            ddl-auto: update
    application:
        name: ORDER-SERVICE
    [illegible]
        [illegible]

eureka:
  instance:
    prefer-ip-address: true
  client:
    fetch-registry: true
    register-with-eureka: true
    service-url:
      defaultZone: ${EUREKA_SERVER_ADDRESS:http://localhost:8761/eureka}
```

test the order service and service registry

3.2 Output

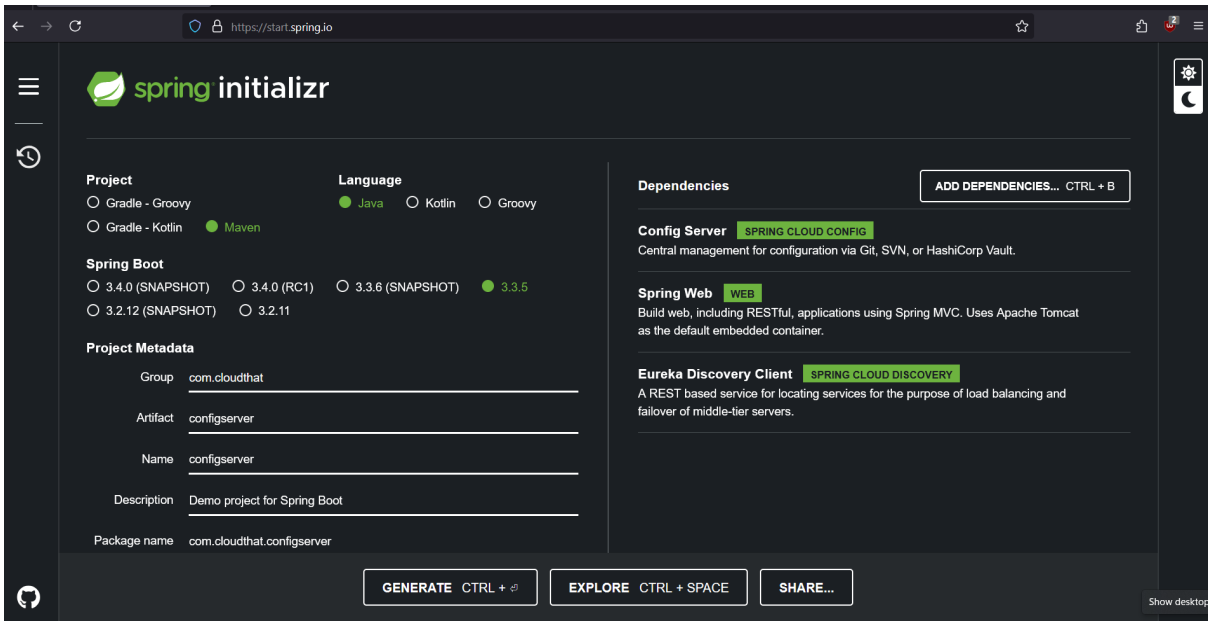


4 Lab: Adding a configuration server & externalize config to github

4.1 Tasks

4.1.1 Task 1: Generating Template from spring initializer

- visit the spring initializer[4]
- Add the dependencies



4.1.2 Task 2: Open the project in intellij

- Convert `application.properties` to `application.yaml`

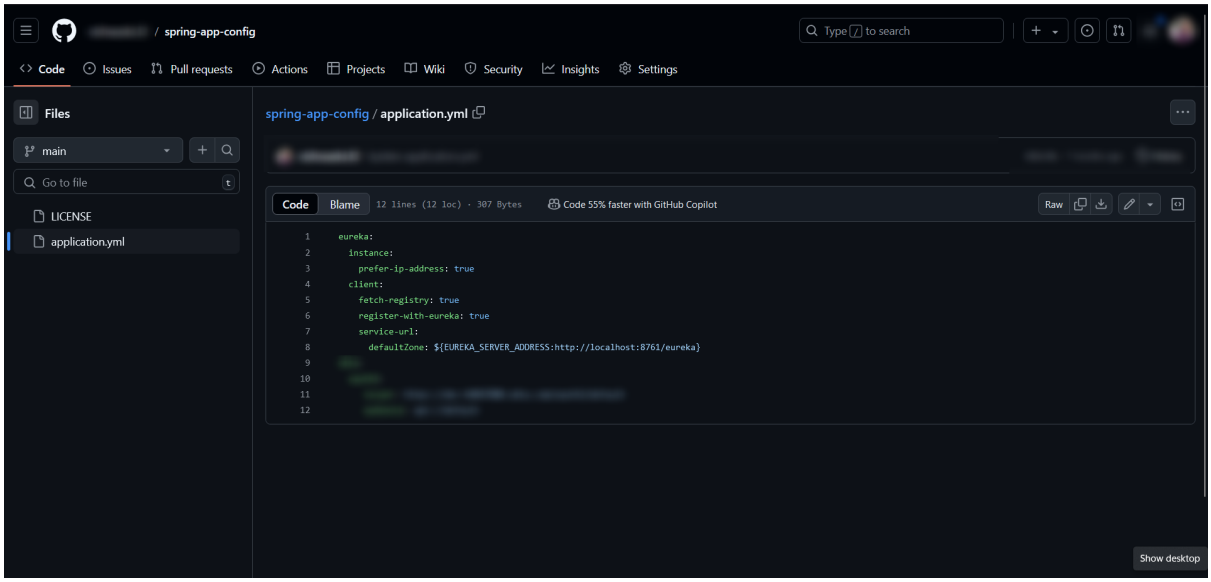
4.1.3 Task 3: Enable Config Server

Add annotation `@EnableConfigServer` in `application.java`

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) { SpringApplication.run(ConfigServerApplication.class, args); }

}
```

[4] https://start.spring.io

4.1.4 Task 4: Add the configuration to github



4.1.5 Task 5: add the configuration to the application.yaml

- In the config Server microservice

```yaml
server:
  port: 9296

spring:
  application:
    name: CONFIG-SERVER
  cloud:
    config:
      server:
        git:
          uri: https://github.com/[redacted]/spring-app-config
          clone-on-start: true


eureka:
  instance:
    prefer-ip-address: true
  client:
    fetch-registry: true
    register-with-eureka: true
    service-url:
      defaultZone: ${EUREKA_SERVER_ADDRESS:http://localhost:8761/eureka}
```

4.1.6 Task 6: Adding config client in product and order service

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

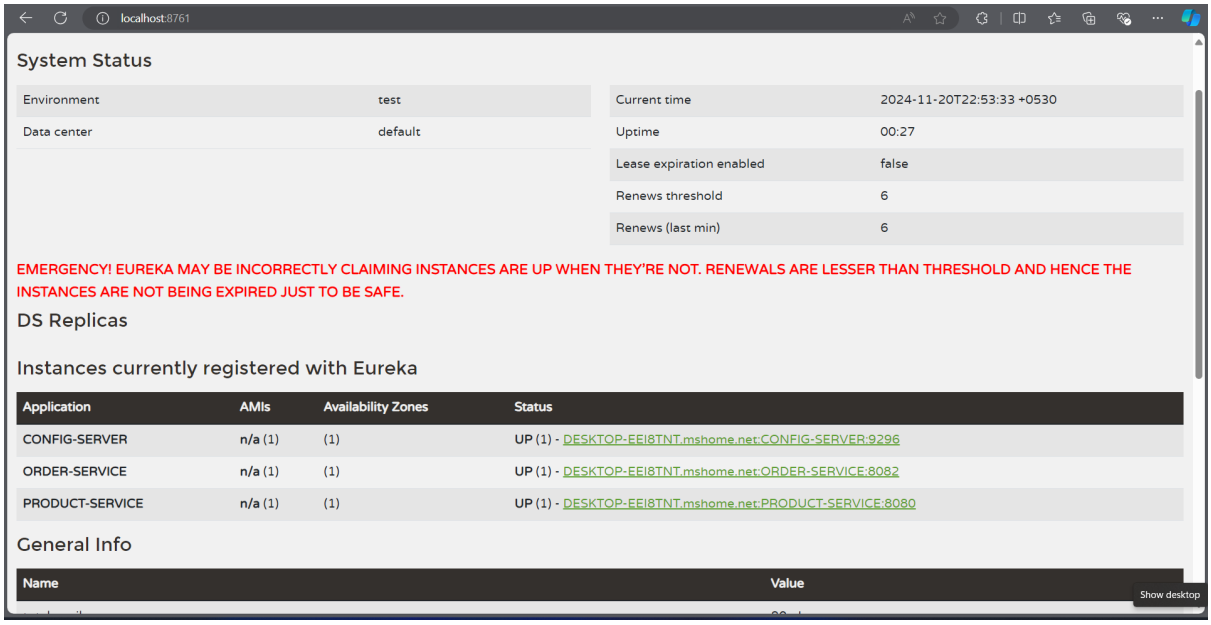
- Then in application.yaml of product service and order service

- Remove/comment eureka configuration
- Add config server configuration

```
spring:
    config:
        import: configserver:http://localhost:9296
```

- Then start service registry, config server and product service and order service

4.2 Output



Visit https://spring.io/guides/gs/centralized-configuration for more

5 Lab: Communication between order service and product service in Ecommerce App

Feign is a declarative web service client. It makes writing web service clients easier. To use Feign create an interface and annotate it. It has pluggable annotation support including Feign annotations and JAX-RS annotations. Feign also supports pluggable encoders and decoders.

5.1 Tasks

5.1.1 Task 1: In the Order Service add the dependency open feign client

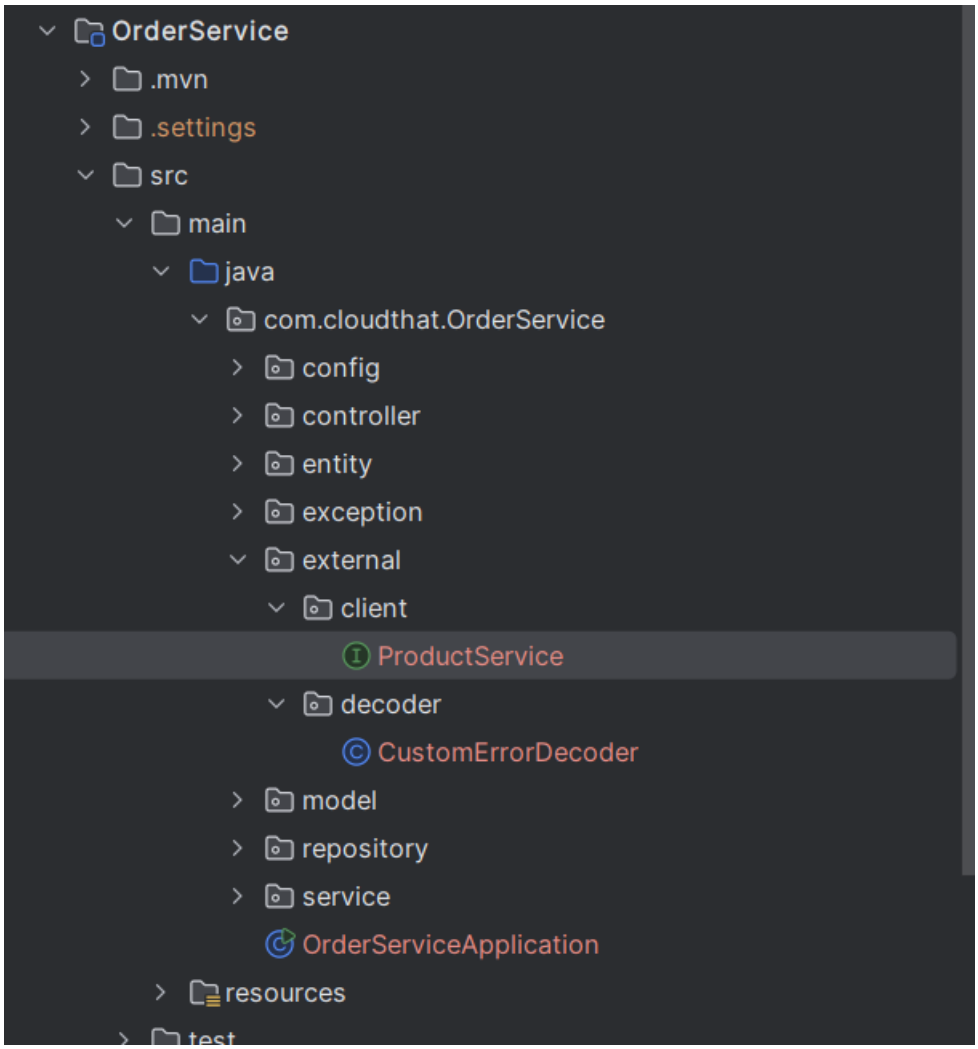
```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

5.1.2 Task 2: Make Order Service as a open feign client

```
7   @SpringBootApplication
8   @EnableFeignClients
9   public class OrderServiceApplication {
10
11      public static void main(String[] args) { SpringApplication.run(OrderServiceApplication.class, args); }
14
15  }
16
```

5.1.3 Task 3: Adding Product service as external dependency

- In the orderService create a `external` package



5.1.4 Task 4: In the interface product service add

- Implement Feign Client(Product Service) in Order service as an external.client package and create an interface
- Declare the method of product service which you are going to use

```
9   @FeignClient(name="PRODUCT-SERVICE/product")  2 usages
10  public interface ProductService {
11      @PutMapping("/reduceQuantity/{id}")  1 usage
12      public ResponseEntity<Void> reduceQuantity(
13              @PathVariable("id") long productId,
14              @RequestParam long quantity
15      );
16  }
17
```

- Add the code to call reduce quantity when order is placed
- Add below code in orderserviceImpl

```
@Override
 public long placeOrder(OrderRequest orderRequest) {
```

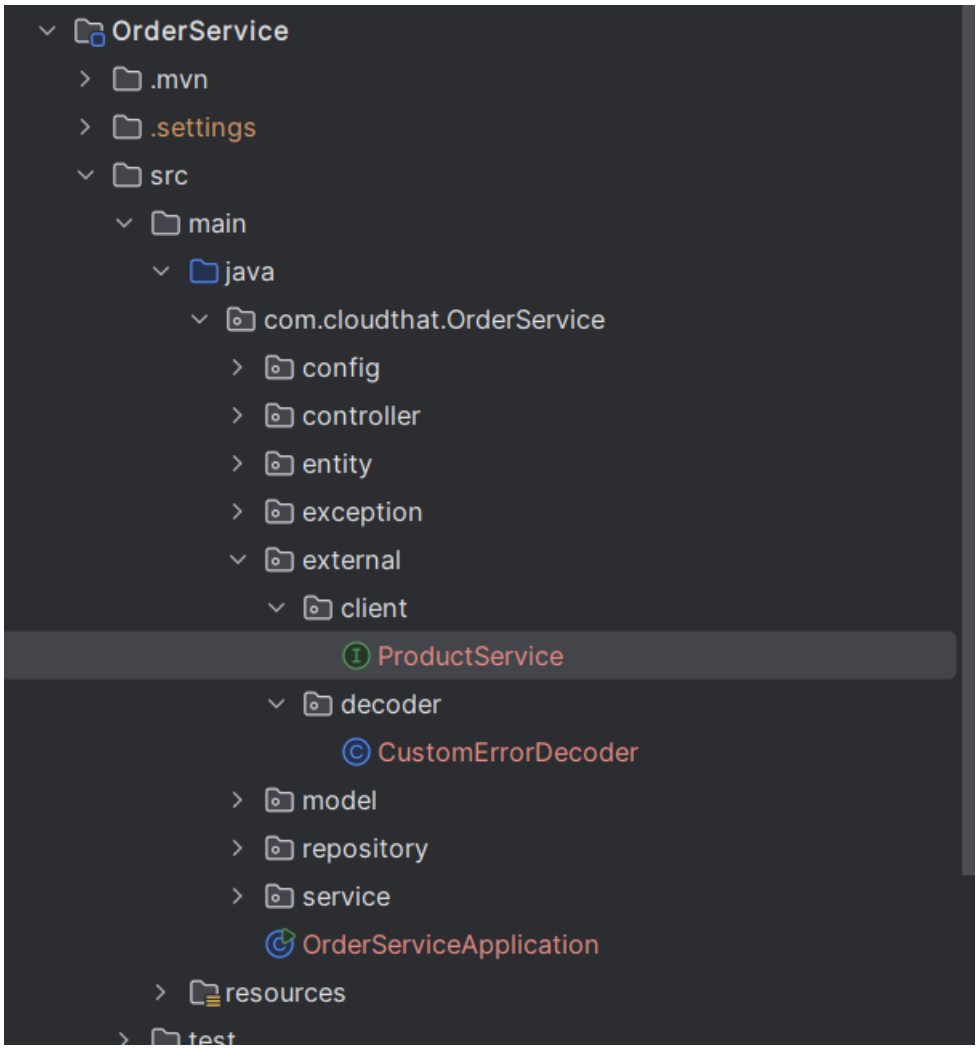
```
        //Order Entity -> Save the data with Status Order Created
        //Product Service - Block Products (Reduce the Quantity)
        //Payment Service -> Payments -> Success-> COMPLETE, Else
        //CANCELLED

        log.info("Placing Order Request: {}", orderRequest);

        productService.reduceQuantity(orderRequest.getProductId(),
            orderRequest.getQuantity());

        log.info("Creating Order with Status CREATED");
        Order order = Order.builder()
                .amount(orderRequest.getTotalAmount())
                .orderStatus("CREATED")
                .productId(orderRequest.getProductId())
                .orderDate(Instant.now())
                .quantity(orderRequest.getQuantity())
                .build();

        order = orderRepository.save(order);

        log.info("Order Places successfully with Order Id: {}",
            order.getId());
        return order.getId();
    }
```

- Restart the order service
- Test the api using place order

6 Lab: Handling errors from one service to another in Eommerce App

6.1 Tasks

6.1.1 Task 1: In the external package – create a package decoder

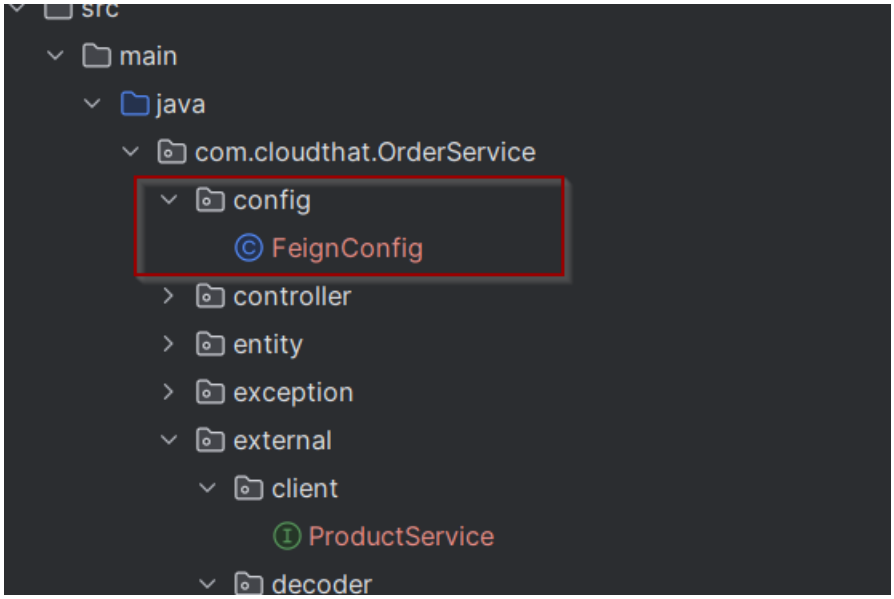


6.1.2 Task 2: Create a class custom error decoder which implements ErrorDecoder from feign client

```
@Log4j2
public class CustomErrorDecoder implements ErrorDecoder {

    @Override
    public Exception decode(String s, Response response) {
        ObjectMapper objectMapper
                = new ObjectMapper();

        log.info("::{}",response.request().url());
        log.info("::{}",response.request().headers());

        try {
```

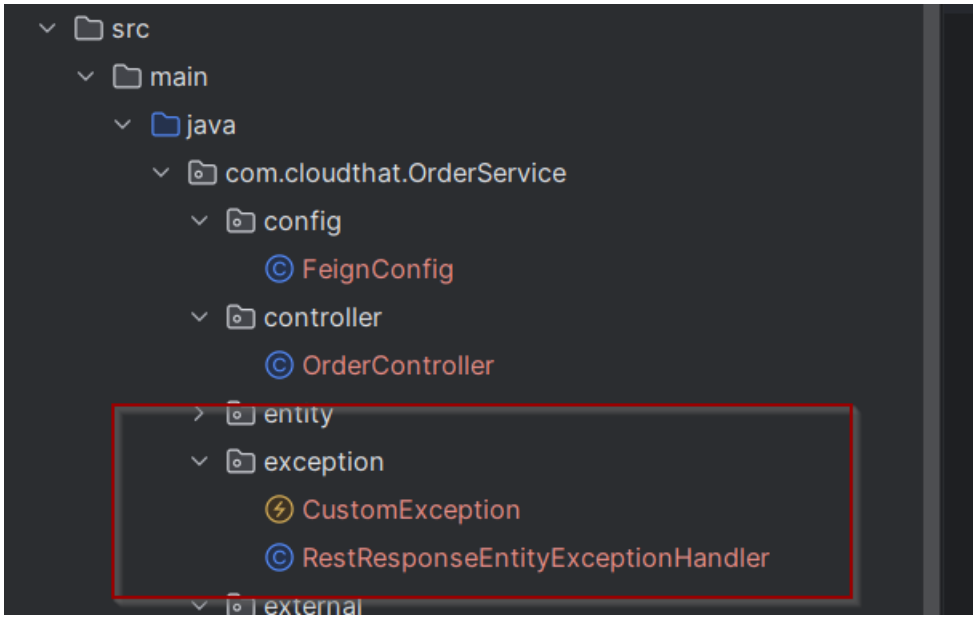
```
            ErrorResponse errorResponse
                    =
                        objectMapper.readValue(response.body().asInputStream(),
                    ErrorResponse.class);

            return new CustomException(errorResponse.getErrorMessage() ,
                    errorResponse.getErrorCode(),
                    response.status());

        } catch (IOException e) {
            throw  new CustomException("Internal Server Error",
                    "INTERNAL_SERVER_ERROR",
                    500);
        }
    }
}
```

6.1.3 Task 3: Add config to use custom error decoder instead of error decoder




```
@Configuration
public class FeignConfig {

    @Bean
    ErrorDecoder errorDecoder() {
        return new CustomErrorDecoder();
    }
}
```

6.1.4 Task 4: Adding Exception Handling in order service



- In `CustomException.java`

```
@Data
public class CustomException extends RuntimeException{

    /**
     *
     */
    private static final long serialVersionUID = 1L;
    private String errorCode;
    private int status;

    public CustomException(String message, String errorCode, int status) {
        super(message);
        this.errorCode = errorCode;
        this.status = status;
    }
}
```

- In `RestResponseEntityExceptionHandler.java`

```
@ControllerAdvice
public class RestResponseEntityExceptionHandler extends
    ResponseEntityExceptionHandler {

    @ExceptionHandler(CustomException.class)
    public ResponseEntity<ErrorResponse>
        handleCustomException(CustomException exception) {
            return new ResponseEntity<ErrorResponse>(new
                ErrorResponse().builder()
                    .errorMessage(exception.getMessage())
                    .errorCode(exception.getErrorCode())
```

```
                        .build(),
                        HttpStatus.valueOf(exception.getStatus()));
    }
}
```

- Create a error response model

```
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class ErrorResponse {
    private String errorMessage;
    private String errorCode;
}
```

7 Lab: Add zipkin tracing for Ecommerce Application

7.1 Tasks

- Visit zipkin quickstart[5]
- You can use the docker version or java version

Docker

The Docker Zipkin project is able to build docker images, provide scripts and a `docker-compose.yml` for launching pre-built images. The quickest start is to run the latest image directly:

```
docker run -d -p 9411:9411 openzipkin/zipkin
```

Java

If you have Java 17 or higher installed, the quickest way to get started is to fetch the latest release as a self-contained executable jar:

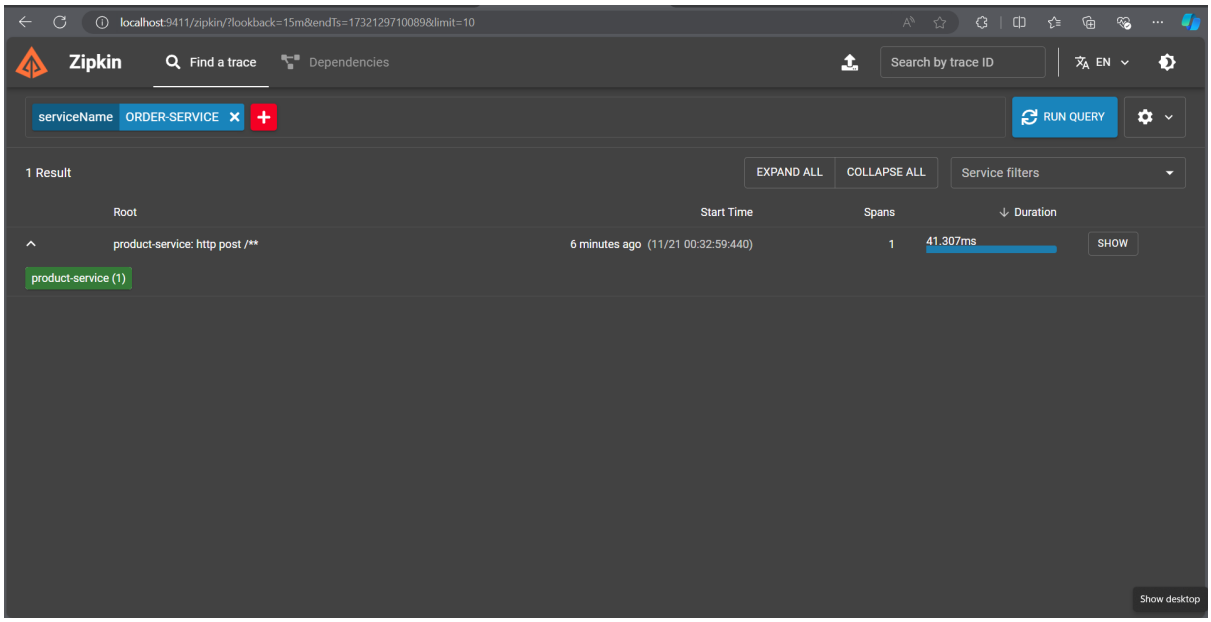
```
curl -sSL https://zipkin.io/quickstart.sh | bash -s
java -jar zipkin.jar
```

- Add Zipkin and Slueth(moved to micrometer from spring 3.0) Dependencies n order and product service

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

You will be able to get trace using trace id and span id

[5] https://zipkin.io/pages/quickstart

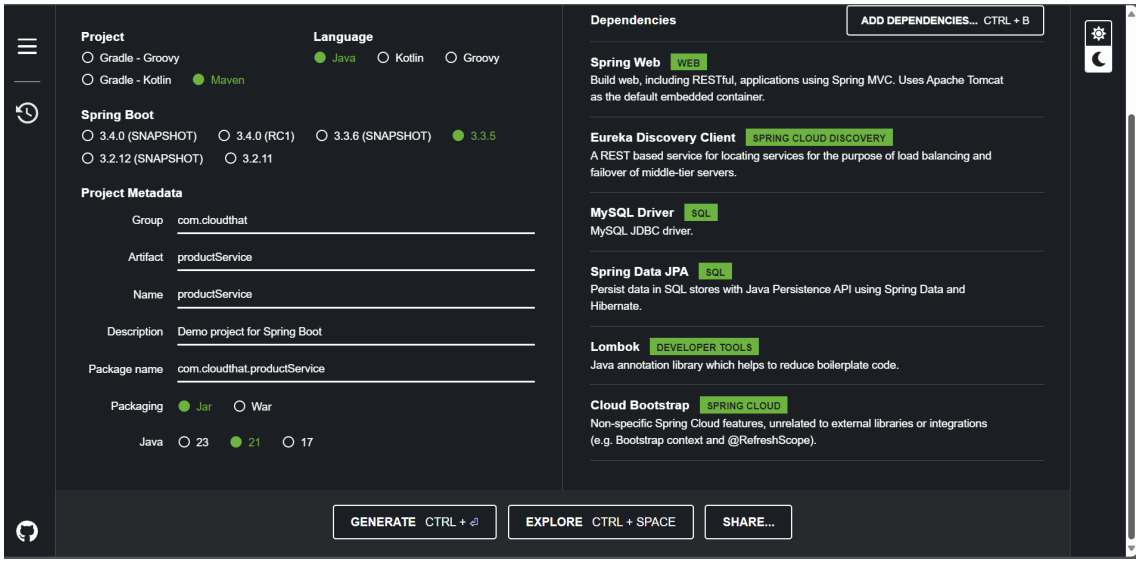

localhost:9411/zipkin/?lookback=15m&endTs=1732129710089&limit=10
Zipkin
Find a trace
Dependencies
Search by trace ID
EN
serviceName
ORDER-SERVICE
RUN QUERY
1 Result
EXPAND ALL
COLLAPSE ALL
Service filters
Root
Start Time
Spans
Duration
product-service: http post /**
6 minutes ago (11/21 00:32:59:440)
1
41.307ms
SHOW
product-service (1)
Show desktop

8 Lab: Adding Payment service to Ecommerce App

8.1 Tasks

8.1.1 Task 1: Generating Template from spring initializer

- visit the spring initializer[6]
- Add the dependencies
  - Spring Web
  - Eureka Client
  - Lombok
  - Data JPA
  - Mysql
  - Cloud Bootstrap
  - zipkin

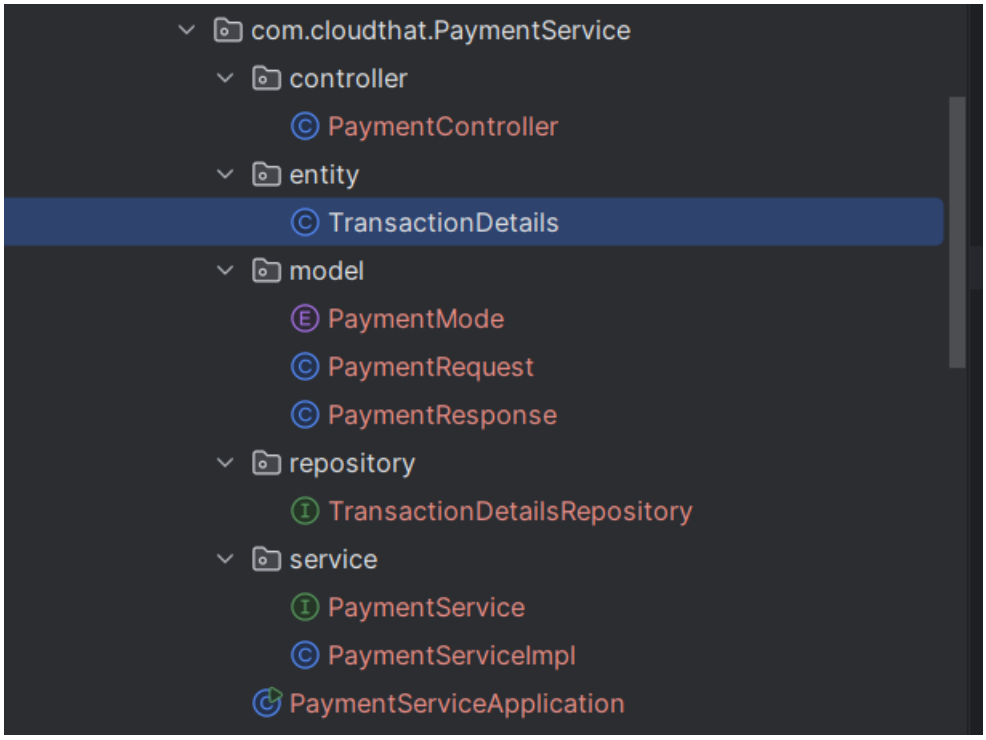


8.1.2 Task 2: Open the project in intelliJ

- Convert `application.properties` to `application.yaml`
- Set `server.port` to `8083`

[6] https://start.spring.io

8.1.3 Task 3: Create Payment Service



- TransactionDetails Entity

```java
@Entity  7 usages
@Table(name = "TRANSACTION_DETAILS")
@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class TransactionDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private long id;

    @Column(name = "ORDER_ID")
    private long orderId;

    @Column(name = "MODE")
    private String paymentMode;

    @Column(name = "REFERENCE_NUMBER")
    private String referenceNumber;

    @Column(name = "PAYMENT_DATE")
    private Instant paymentDate;

    @Column(name = "STATUS")
    private String paymentStatus;

    @Column(name = "AMOUNT")
    private long amount;
}
```

- PaymentMode

```java
public enum PaymentMode {  4 usages
    CASH,  no usages
    PAYPAL,  no usages
    DEBIT_CARD,  no usages
    CREDIT_CARD,  no usages
    APPLE_PAY  no usages
}
```

- With methods doPayment

```java
@Override  1 usage
public long doPayment(PaymentRequest paymentRequest) {
    log.info( message: "Recording Payment Details: {}", paymentRequest);

    TransactionDetails transactionDetails
            = TransactionDetails.builder()
            .paymentDate(Instant.now())
            .paymentMode(paymentRequest.getPaymentMode().name())
            .paymentStatus("SUCCESS")
            .orderId(paymentRequest.getOrderId())
            .referenceNumber(paymentRequest.getReferenceNumber())
            .amount(paymentRequest.getAmount())
            .id().build();

    transactionDetailsRepository.save(transactionDetails);

    log.info( message: "Transaction Completed with Id: {}", transactionDetails.getId());

    return transactionDetails.getId();
}
```

- Get Payment details by order id

```java
@Override  1 usage
public PaymentResponse getPaymentDetailsByOrderId(String orderId) {
    log.info( message: "Getting payment details for the Order Id: {}", orderId);

    TransactionDetails transactionDetails
            = transactionDetailsRepository.findByOrderId(Long.valueOf(orderId));

    PaymentResponse paymentResponse
            = PaymentResponse.builder()
            .paymentId(transactionDetails.getId())
            .paymentMode(PaymentMode.valueOf(transactionDetails.getPaymentMode()))
            .paymentDate(transactionDetails.getPaymentDate())
            .orderId(transactionDetails.getOrderId())
            .status(transactionDetails.getPaymentStatus())
            .amount(transactionDetails.getAmount())
            .build();

    return paymentResponse;
}
```

9 Lab: Communication among services using RestTemplate

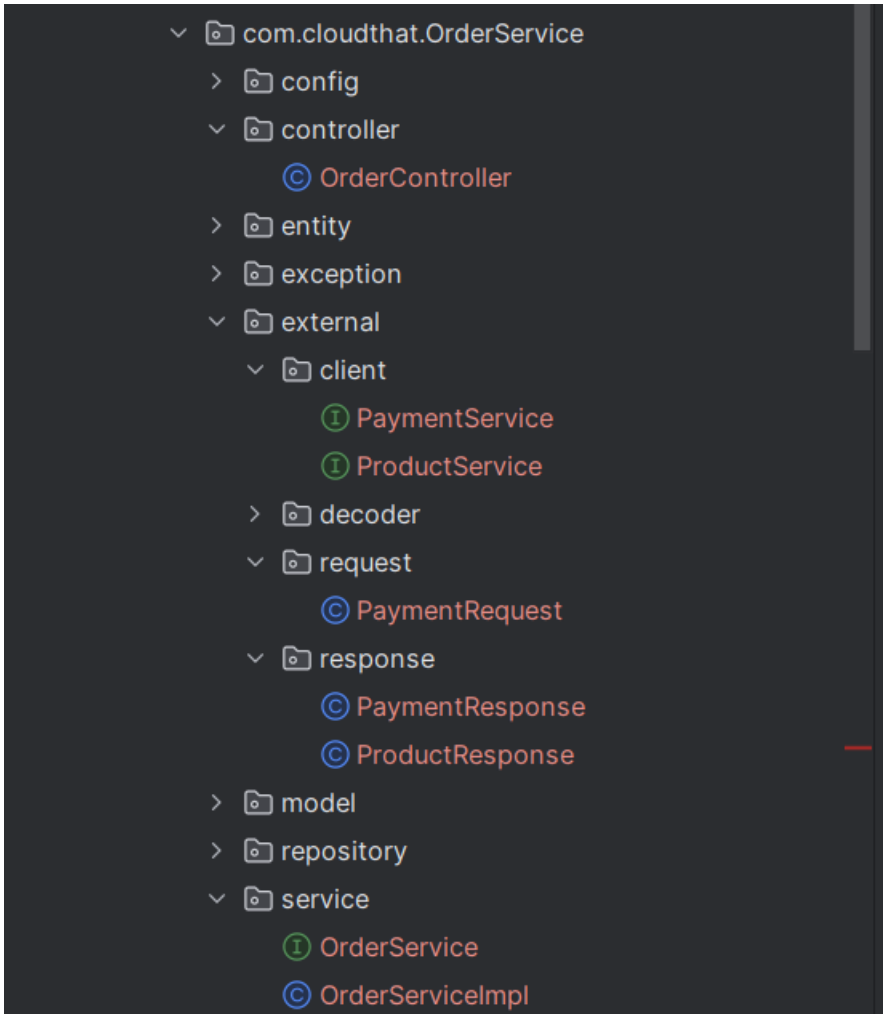
9.1 Tasks

9.1.1 Task 1: In the Service you want to fetch data ORDER SERVICE

```
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication {

    public static void main(String[] args) { SpringApplication.run(OrderServiceApplication.class, args); }

    @Bean  no usages
    @LoadBalanced
    public RestTemplate restTemplate() { return new RestTemplate(); }
}
```

9.1.2 Task 2: In the external package

- Create Client



9.1.3 Task 5: Create Payment service openfeign client

```
@FeignClient(name = "PAYMENT-SERVICE/payment")
public interface PaymentService {

    @PostMapping
    public ResponseEntity<Long> doPayment(@RequestBody PaymentRequest
       paymentRequest);

    default ResponseEntity<Long> fallback(Exception e) {
        throw new CustomException("Payment Service is not available",
                "UNAVAILABLE",
                500);
    }
}
```

This can also be implemented using resttemplate `postforobject` or `postforentity`

9.1.4 Task 4: Modify PlaceOrder Method to updated payment

```
    @Override
    public long placeOrder(OrderRequest orderRequest) {

        //Order Entity -> Save the data with Status Order Created
        //Product Service - Block Products (Reduce the Quantity)
        //Payment Service -> Payments -> Success-> COMPLETE, Else
        //CANCELLED



        productService.reduceQuantity(orderRequest.getProductId(),
           orderRequest.getQuantity());


        Order order = Order.builder()
                .amount(orderRequest.getTotalAmount())
                .orderStatus("CREATED")
                .productId(orderRequest.getProductId())
                .orderDate(Instant.now())
                .quantity(orderRequest.getQuantity())
                .build();

        order = orderRepository.save(order);

               PaymentRequest paymentRequest
                = PaymentRequest.builder()
                .orderId(order.getId())
                .paymentMode(orderRequest.getPaymentMode())
                .amount(orderRequest.getTotalAmount())
                .build();
```

```
        String orderStatus = null;
        try {
            paymentService.doPayment(paymentRequest);

            orderStatus = "PLACED";
        } catch (Exception e) {

            orderStatus = "PAYMENT_FAILED";
        }
        order.setOrderStatus(orderStatus);
        orderRepository.save(order);


        return order.getId();
    }
```

9.1.5 Task 5: Implement getorder details

```
public OrderResponse getOrderDetails(long orderId) {

        Order order
                = orderRepository.findById(orderId)
                .orElseThrow(() -> new CustomException("Order not found
                    for the order Id:" + orderId,
                        "NOT_FOUND",
                        404));


        ProductResponse productResponse =
            restTemplate.getForObject("http://PRODUCT-SERVICE/product/" +
            order.getProductId(), ProductResponse.class);


        PaymentResponse paymentResponse
                = restTemplate.getForObject(
                        "http://PAYMENT-SERVICE/payment/order/" +
                            order.getId(),
                PaymentResponse.class
                );

        OrderResponse.ProductDetails productDetails
                = OrderResponse.ProductDetails
                .builder()
                .productName(productResponse.getProductName())
                .productId(productResponse.getProductId())
                .build();

        OrderResponse.PaymentDetails paymentDetails
                = OrderResponse.PaymentDetails
                .builder()
```

```
                .paymentId(paymentResponse.getPaymentId())
                .paymentStatus(paymentResponse.getStatus())
                .paymentDate(paymentResponse.getPaymentDate())
                .paymentMode(paymentResponse.getPaymentMode())
                .build();

        OrderResponse orderResponse
                = OrderResponse.builder()
                .orderId(order.getId())
                .orderStatus(order.getOrderStatus())
                .amount(order.getAmount())
                .orderDate(order.getOrderDate())
                .productDetails(productDetails)
                .paymentDetails(paymentDetails)
                .build();

        return orderResponse;
    }
```

9.2 Output

Test the endpoints with relevant details

Refer :

1. https://spring.io/guides/gs/consuming-rest
2. https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/client/RestTemplate.html
3. https://www.springcloud.io/post/2022-03/spring-resttemplate/#gsc.tab=0

10 Lab: Implementing Logging in Ecommerce Application

10.1 Tasks

10.1.1 Task 1: Enable Log4J2 in application

Since we had already included Lombok as dependency on our projects

use `@Log4j2` on ProductService, OrderService etc..

Example:

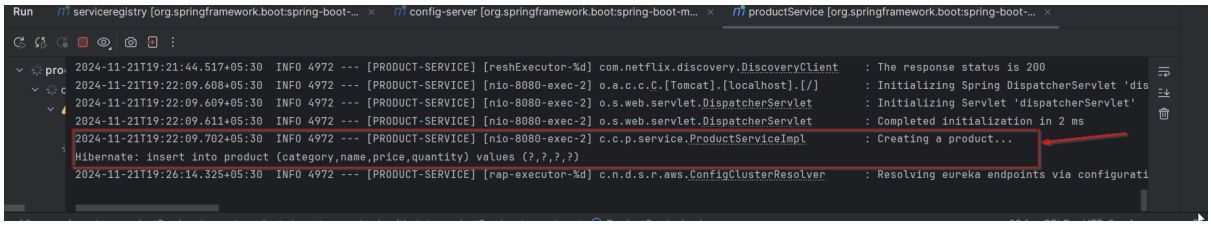
```
@Service  no usages
@Log4j2
public class ProductServiceImpl implements ProductService{

    @Autowired  4 usages
    private ProductRepository productRepository;

    @Override  1 usage
    public long addProduct(ProductRequest productRequest) {
        log.info("Adding Product..");

        Product product
                = Product.builder()
                .productName(productRequest.getName())
                .quantity(productRequest.getQuantity())
                .price(productRequest.getPrice())
                .productId().build();

        productRepository.save(product);

        log.info("Product Created");
        return product.getProductId();
    }
```

10.1.2 Task 2: Trigger endpoint

Trigger the endpoint in postman tool

10.2 Output

View the log in terminal



Similarly add the log for info, debug, error whereever necessary in application which would be helpful for auditing.

11 Lab: Adding API Gateway to Ecommerce Application

11.1 Tasks

11.1.1 Task 1: Generate template

visit spring starter[7]

11.1.2 Task 2: Add the dependencies

- cloud bootstrap,
- gateway,
- spring reactive web,
- zipkin,
- Lombok,
- config client,
- eureka client,
- spring boot actuator

```
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>
```

The above is a reactive gateway

11.1.3 Task 3: Open in IntelliJ

- Convert from `application.properties` to `application.yml`
- Add the below configuration to `application.yml`

```yml
server:
  port: 9090
spring:
  application:
    name: API-GATEWAY
  config:
    import: configserver:http://localhost:9296
```

11.1.4 Task 4: Add the routes

```yml
spring:
  application:
    name: API-GATEWAY
  config:
```

[7] https://start.spring.io

```yaml
    import:
      - configserver:http://localhost:9296
  cloud:
    gateway:
      mvc:
        routes:
          - id: ORDER-SERVICE
            uri: lb://ORDER-SERVICE
            predicates:
              - Path=/order/**

          - id: PAYMENT-SERVICE
            uri: lb://PAYMENT-SERVICE
            predicates:
              - Path=/payment/**

          - id: PRODUCT-SERVICE
            uri: lb://PRODUCT-SERVICE
            predicates:
              - Path=/product/**
    discovery:
      enabled: true
```

From now on change all routes to route from `localhost:9090`

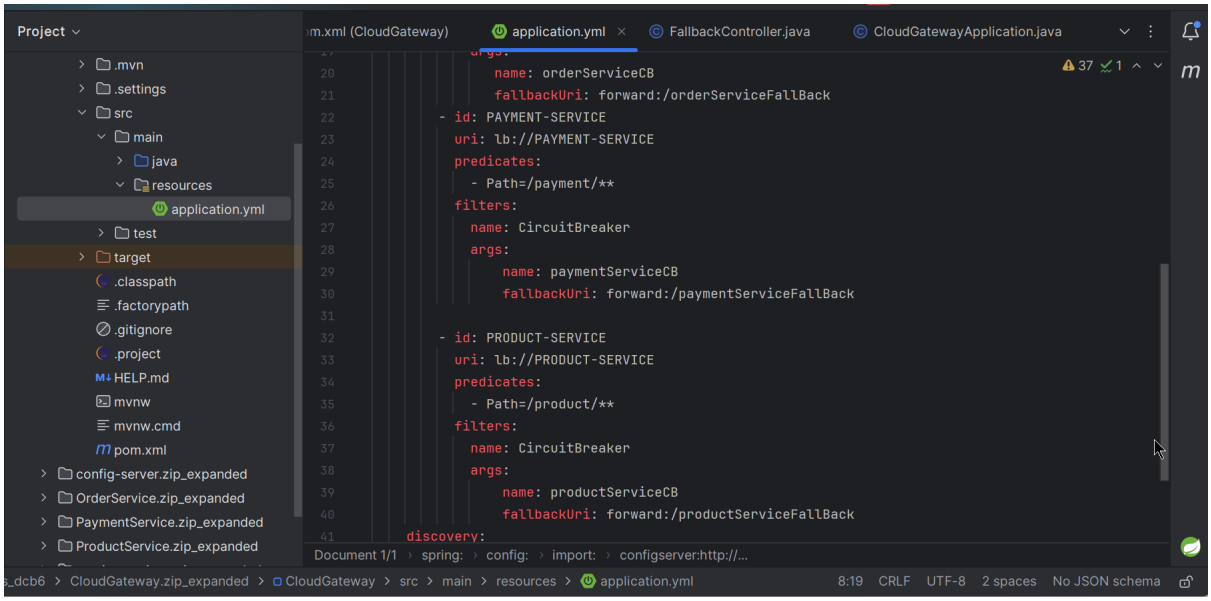
12 Lab: Implementing Circuit Breaker

12.1 Tasks

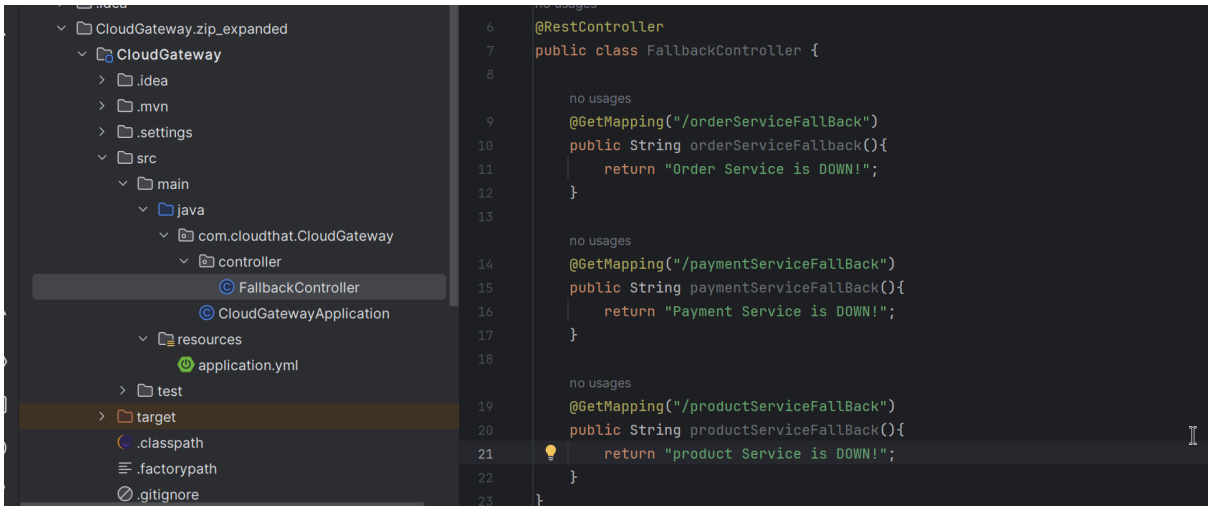
12.1.1 Task 1: Adding dependency to cloud gateway

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

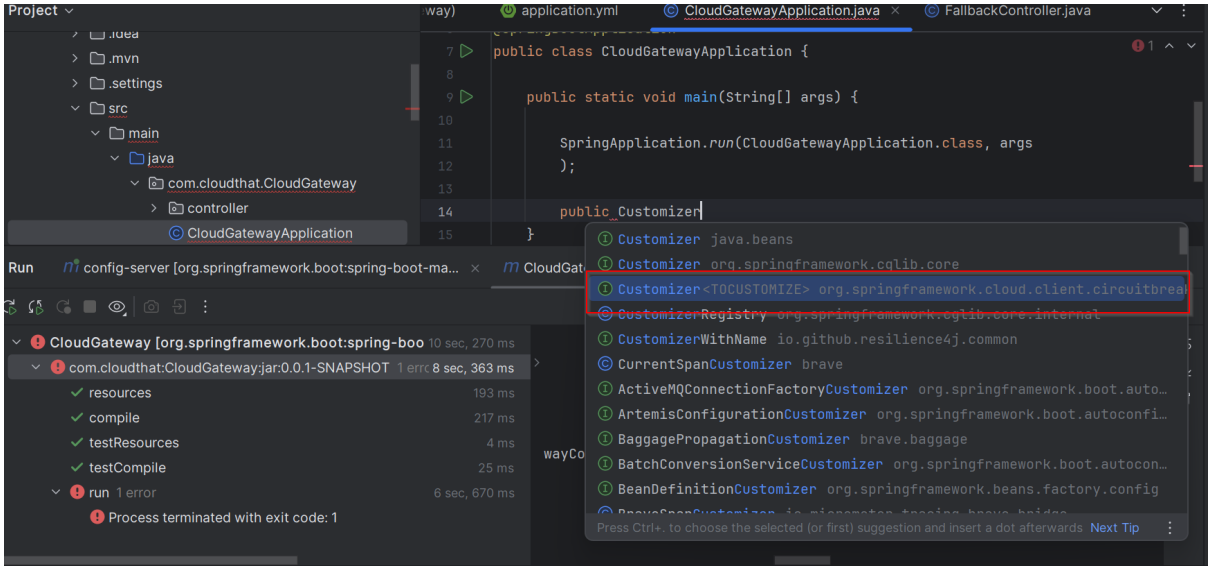
12.1.2 Task 2: Add fallback uri



12.1.3 Task 3: Implement Fallback controller

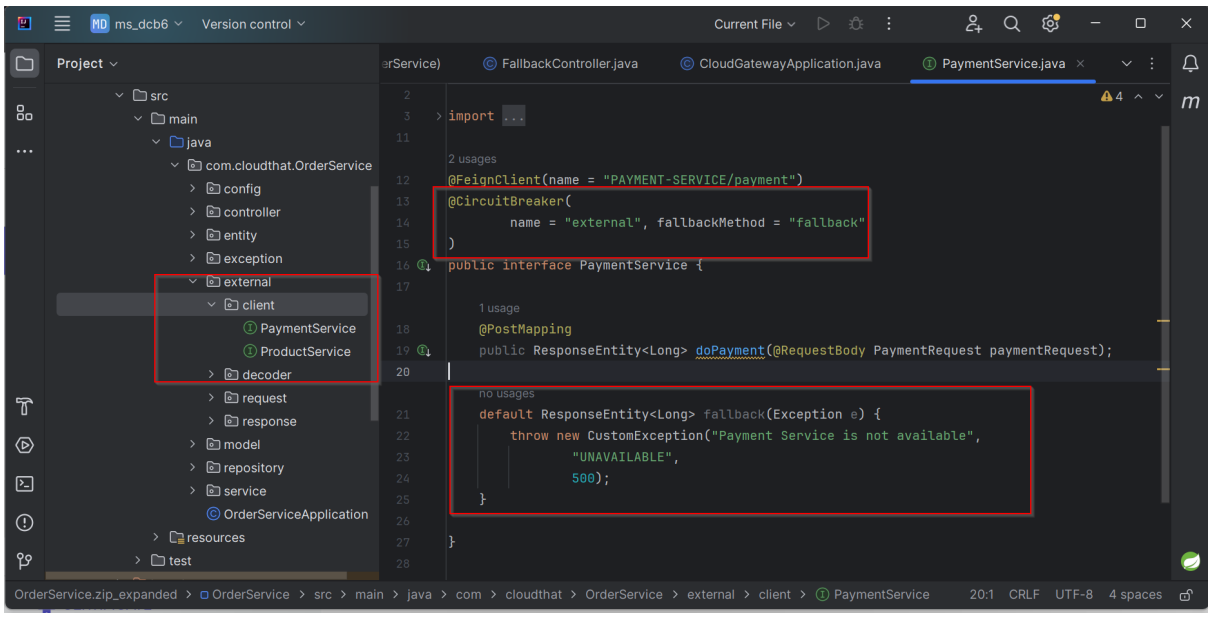


12.1.4 Task 4: In cloudgateway – create bean for circuit breaker



12.1.5 Task 5: Adding Circuitbreaker

In Order service pom.xml add circuitbreaker dependency In feign clients insert circuit breaker



12.1.6 Task 6: Add configuration in order service

```
plication.java   PaymentService.java   ProductService.java   OrderService\...\application.yml
20      web:
21        exposure:
22          include: '*'
23    tracing:
24      sampling:
25        probability: 1.0
26
27  resilience4j:
28    circuitbreaker:
29      instances:
30        external:
31          event-consumer-buffer-size: 10
32          failure-rate-threshold: 50
33          minimum-number-of-calls: 5
34          automatic-transition-from-open-to-half-open-enabled: true
35          wait-duration-in-open-state: 5s
36          permitted-number-of-calls-in-half-open-state: 3s
37          sliding-window-size: 10
38          sliding-window-type: COUNT_BASED
39
40  #eureka:
41  #  instance:
Document 1/1
```

```yaml
resilience4j:
  circuitbreaker:
    instances:
      external:
        event-consumer-buffer-size: 10
        failure-rate-threshold: 50
        minimum-number-of-calls: 5
        automatic-transition-from-open-to-half-open-enabled: true
        wait-duration-in-open-state: 5s
        permitted-number-of-calls-in-half-open-state: 3
        sliding-window-size: 10
        sliding-window-type: COUNT_BASED
```

12.1.7 Task 7: Implementing Rate Limiter

- Implementing Rate Limiter using Redis and Resilience4J
- Download and start redis docker image
- Add dependency of reactive redis In cloud gateway

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

```
src
  main
    java
      com.cloudthat.CloudGateway
        controller
          FallbackController
          CloudGatewayApplication
    resources
  test
target
.classpath
.factorypath
.gitignore
.project
HELP.md
mvnw
mvnw.cmd
pom.xml
config-server.zip_expanded
OrderService.zip_expanded
PaymentService.zip_expanded
ProductService.zip_expanded
```

FallbackController.java | CloudGatewayApplication.java | pom.xml (CloudGateway)

```
3   import ...
16
17  @SpringBootApplication
18  public class CloudGatewayApplication {
19
20      public static void main(String[] args) {
21
22          SpringApplication.run(CloudGatewayApplication.class, args);
23      }
24
        no usages
25      @Bean
26      public KeyResolver userKeyResolver(){
27          return exchange -> Mono.just( data: "userKey");
28      }
29
        no usages
30      @Bean
31      public Customizer<ReactiveResilience4JCircuitBreakerFactory> defaultCustomizer(
32          return factory -> factory.configureDefault(id -> new Resilience4JConfigBuil
33                  .circuitBreakerConfig(CircuitBreakerConfig.ofDefaults())
34                  .timeLimiterConfig(TimeLimiterConfig.custom().timeoutDuration(Durat
35      }
```

12.1.8 Task 8: Add replenish rate and burst capacity

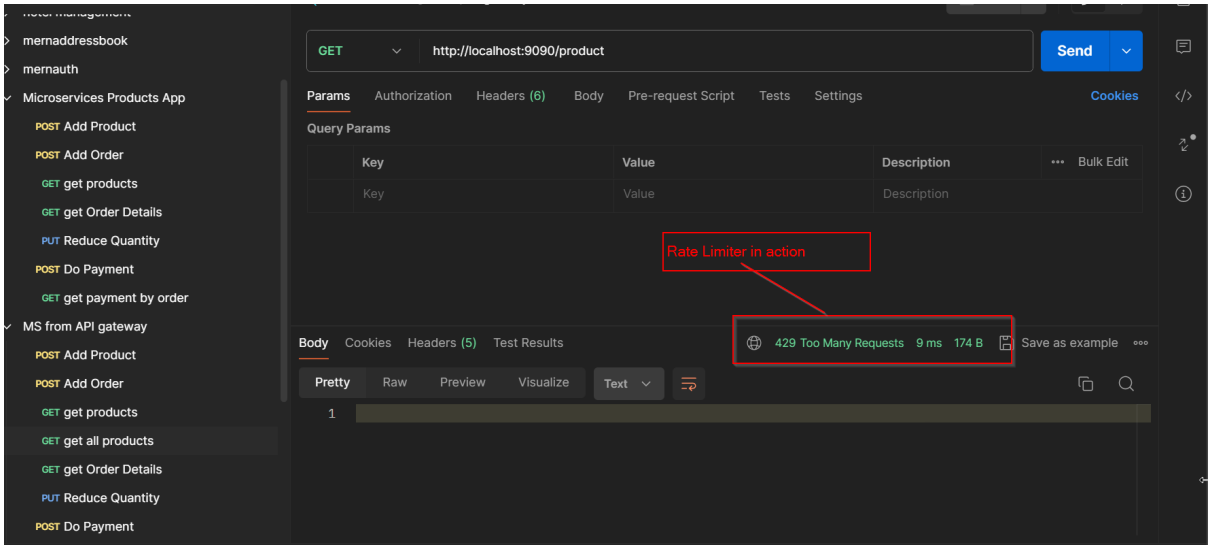
application.yml | mvnw.cmd | pom.xml (CloudGateway)

```yaml
        uri: lb://PAYMENT-SERVICE
        predicates:
          - Path=/payment/**
        filters:
          - name: CircuitBreaker
            args:
              name: paymentServiceCB
              fallbackUri: forward:/paymentServiceFallBack
          - name: RequestRateLimiter
            args:
              redis-rate-limiter.replenishRate: 1
              redis-rate-limiter.burstCapacity: 1
              redis-rate-limiter.requestedTokens: 1

      - id: PRODUCT-SERVICE
        uri: lb://PRODUCT-SERVICE
        predicates:
          - Path=/product/**
        filters:
          - name: CircuitBreaker
            args:
              name: productServiceCB
```

Add this for every service in cloud gateway

```yaml
        - id: PAYMENT-SERVICE
          uri: lb://PAYMENT-SERVICE
          predicates:
            - Path=/payment/**
          filters:
            - name: CircuitBreaker
              args:
                name: paymentServiceCB
                fallbackUri: forward:/paymentServiceFallBack
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 1
                redis-rate-limiter.burstCapacity: 1
                redis-rate-limiter.requestedTokens: 1
```

12.2 Output



Refer

1. https://resilience4j.readme.io/docs/getting-started-3
2. https://spring.io/guides/gs/cloud-circuit-breaker